

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO12 – Develop Program Using Class, Module and Package

Nominal Duration: Theory 3.5 Hours / Practical 6 Hours

L.O 12: DEVELOP PROGRAM USING CLASS, MODULE AND PACKAGE

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Explain the basic concepts of Object-Oriented Programming (OOP).
2. Identify classes, objects, attributes, and methods.
3. Create Python classes using correct syntax.
4. Apply the `__init__()` constructor to initialize objects.
5. Define instance variables and class variables.
6. Define and invoke class methods according to program requirements.
7. Create and instantiate objects from classes.
8. Access object attributes.
9. Modify object attributes.
10. Call object methods.
11. Explain the purpose of Python modules.
12. Identify built-in and user-defined modules.
13. Create reusable modules containing functions and classes.
14. Import modules using `import`.
15. Import specific components using `from ... import`.
16. Apply module aliases using `import ... as`.
17. Execute functions and instantiate classes from imported modules.
18. Explain Python package structure.
19. Create packages for organizing related modules.
20. Import modules and components from packages.
21. Apply Python project file and folder organization.
22. Apply basic PEP 8 coding and naming conventions.
23. Integrate classes, modules, and packages into a complete Python application.
24. Test and debug modular Python applications.
25. Apply code reusability and modular programming principles.

INFORMATION SHEET No. 12.1

Title: Understanding Python Classes, Modules, and Packages

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain basic Object-Oriented Programming concepts.
- Define classes and objects.
- Explain attributes and methods.
- Create Python classes.
- Apply the `__init__()` constructor.
- Explain instance and class variables.
- Create and use objects.
- Access and modify object attributes.
- Explain Python modules.
- Create user-defined modules.
- Import built-in and user-defined modules.
- Apply different import statements.
- Explain Python packages.
- Create a basic package structure.
- Import modules from packages.
- Explain code reusability and modular programming.
- Apply basic PEP 8 conventions.
- Organize Python project files appropriately.
- Test and debug class-, module-, and package-based applications.

1. Introduction to Object-Oriented Programming

Object-Oriented Programming, commonly called **OOP**, is a programming approach that organizes programs around objects.

Objects can contain:

- Data
- Attributes
- Behaviors
- Methods

For example, in a student-management program, a student may have:

Name
Age
Course
Score

and may perform behaviors such as:

Display information
Update score
Calculate result

A Python class can represent this student.

2. What Is a Class?

A class is a blueprint or template for creating objects.

Example:

```
class Student:  
    pass
```

This creates a class named:

Student

At this point, the class does not contain attributes or methods.

3. What Is an Object?

An object is an instance created from a class.

Example:

```
class Student:  
    pass
```

```
student1 = Student()
```

Here:

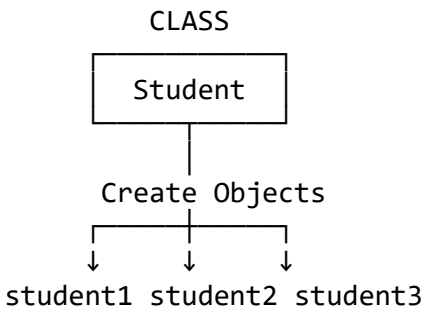
Student → Class
student1 → Object

The class describes the structure.

The object represents an actual instance of that structure.

4. Class and Object Relationship

Conceptually:



One class can create many objects.

5. Basic Class Syntax

Syntax:

```
class ClassName:
    statements
```

Example:

```
class Student:
    pass
```

Python class names normally use **CapWords/PascalCase** naming.

Recommended:

```
class StudentRecord:
    pass
```

6. Attributes

An attribute stores information associated with an object.

Example attributes for a student include:

```
name
age
course
score
```

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

In this example:

```
self.name
self.age
```

are instance attributes.

7. The `init()` Constructor

Python commonly uses the special method:

```
__init__()
```

to initialize new objects.

Syntax:

```
class ClassName:
    def __init__(self, parameters):
        statements
```

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

When an object is created:

```
student1 = Student("Dara", 20)
```

Python calls `__init__()` to initialize the object's data.

8. The `self` Parameter

`self` refers to the current object.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

The statement:

```
self.name = name
```

stores the value supplied through name in the current object's name attribute.

9. Creating an Object

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
student1 = Student("Dara", 20)
```

The object:

```
student1
```

now contains:

```
name = Dara
age = 20
```

10. Accessing Object Attributes

Use dot notation.

Syntax:

```
object.attribute
```

Example:

```
print(student1.name)
print(student1.age)
```

Output:

```
Dara
20
```

11. Modifying Object Attributes

Object attributes can be modified.

Example:

```
student1.age = 21  
  
print(student1.age)
```

Output:

21

12. Methods

A method is a function defined inside a class.

Example:

```
class Student:  
    def __init__(self, name):  
        self.name = name  
  
    def display_name(self):  
        print("Name:", self.name)
```

Call the method:

```
student1 = Student("Dara")  
  
student1.display_name()
```

Output:

Name: Dara

13. Methods with Parameters

Methods can receive additional parameters.

Example:

```
class Student:  
    def __init__(self, name):  
        self.name = name  
  
    def update_name(self, new_name):  
        self.name = new_name
```

Use:

```
student1 = Student("Dara")  
  
student1.update_name("Sokha")
```

```
print(student1.name)
```

Output:

Sokha

14. Methods Returning Values

A method can return a value.

Example:

```
class Student:
    def __init__(self, score):
        self.score = score

    def get_result(self):
        if self.score >= 50:
            return "Pass"
        else:
            return "Fail"
```

Use:

```
student1 = Student(85)
```

```
print(student1.get_result())
```

Output:

Pass

15. Instance Variables

Instance variables belong to individual objects.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Create:

```
student1 = Student("Dara")
student2 = Student("Sokha")
```

Then:

```
student1.name → Dara  
student2.name → Sokha
```

Each object has its own value.

16. Class Variables

A class variable is shared at the class level.

Example:

```
class Student:  
    school = "Technical Training Center"  
  
    def __init__(self, name):  
        self.name = name
```

Use:

```
student1 = Student("Dara")  
student2 = Student("Sokha")
```

```
print(student1.school)  
print(student2.school)
```

Output:

```
Technical Training Center  
Technical Training Center
```

17. Instance Variable vs Class Variable

Feature	Instance Variable	Class Variable
Defined	Usually through self	In class body
Belongs to	Individual object	Class
Can differ per object	Yes	Normally shared
Example	self.name	school

18. Complete Basic Class Example

```
class Student:  
    school = "Technical Training Center"  
  
    def __init__(self, student_id, name, score):  
        self.student_id = student_id  
        self.name = name
```

```

        self.score = score

    def display_info(self):
        print("ID:", self.student_id)
        print("Name:", self.name)
        print("Score:", self.score)

    def get_result(self):
        if self.score >= 50:
            return "Pass"
        return "Fail"

```

Create an object:

```
student1 = Student("ST001", "Dara", 85)
```

Call methods:

```
student1.display_info()
print("Result:", student1.get_result())
```

19. Creating Multiple Objects

Example:

```

student1 = Student("ST001", "Dara", 85)
student2 = Student("ST002", "Sokha", 90)
student3 = Student("ST003", "Vanna", 45)

```

Each object has independent instance data.

20. Advantages of Classes and Objects

Classes help programmers:

- Model real-world entities.
- Group related data and behavior.
- Improve program organization.
- Reduce repeated code.
- Create reusable components.
- Manage larger programs.
- Support modular development.

21. Introduction to Python Modules

A module is a Python file containing reusable Python code.

A module may contain:

- Variables
- Functions
- Classes
- Constants
- Other statements

A Python module normally has the extension:

`.py`

Example:

`calculator.py`

22. Purpose of Modules

Modules help to:

- Divide large programs into smaller files.
- Reuse code.
- Organize related functions and classes.
- Improve maintainability.
- Simplify testing.
- Reduce duplicated code.

23. Built-In Modules

Python provides many modules as part of its standard library.

Examples include:

`math`
`random`
`datetime`
`os`
`sys`

Example:

```
import math
```

```
print(math.sqrt(25))
```

Output:

5.0

24. Importing a Built-In Module

Basic syntax:

```
import module_name
```

Example:

```
import math
```

```
print(math.pi)
```

25. Using from ... import

A specific function or class can be imported.

Example:

```
from math import sqrt
```

```
print(sqrt(81))
```

Output:

9.0

26. Using import ... as

A module can be given an alias.

Example:

```
import math as m
```

```
print(m.sqrt(16))
```

Output:

4.0

27. Importing a Specific Function with an Alias

Example:

```
from math import sqrt as square_root

print(square_root(100))
```

Output:

10.0

28. Creating a User-Defined Module

Create a file named:

calculator.py

Enter:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Create another file:

main.py

Use:

```
import calculator

print(calculator.add(5, 10))
print(calculator.subtract(20, 5))
```

Output:

15
15

29. Importing Specific Functions from a User-Defined Module

Given:

calculator.py

Use:

```
from calculator import add  
  
print(add(5, 10))
```

30. Importing with an Alias

Example:

```
import calculator as calc  
  
print(calc.add(10, 20))
```

Output:

30

31. Creating a Module Containing a Class

Create:

student.py

Code:

```
class Student:  
    def __init__(self, name, score):  
        self.name = name  
        self.score = score  
  
    def display_info(self):  
        print("Name:", self.name)  
        print("Score:", self.score)
```

In:

main.py

use:

```
from student import Student  
  
student1 = Student("Dara", 85)  
  
student1.display_info()
```

32. Module File Organization

Example:

```
python_project/  
├── main.py  
├── calculator.py  
└── student.py
```

Each file has a specific purpose.

33. Introduction to Python Packages

A package organizes related modules into a directory.

Example structure:

```
student_project/  
├── main.py  
└── school/  
    ├── __init__.py  
    ├── student.py  
    └── utilities.py
```

The folder:

`school`

acts as the package.

For beginner projects, an `__init__.py` file is commonly included to make the package structure explicit.

34. Purpose of Packages

Packages help:

- Organize related modules.
- Prevent file-name conflicts.
- Improve project structure.
- Support larger applications.
- Improve code reuse.
- Separate application responsibilities.

35. The `__init__.py` File

A commonly used package contains:

`__init__.py`

Example:

```
school/  
├── __init__.py  
├── student.py  
└── utilities.py
```

The file may be empty for a basic package.

36. Importing a Module from a Package

Suppose:

```
school/  
└── utilities.py
```

contains:

```
def calculate_average(scores):  
    return sum(scores) / len(scores)
```

Import:

```
from school import utilities  
  
print(utilities.calculate_average([80, 90, 70]))
```

Output:

80.0

37. Importing a Function from a Package Module

Use:

```
from school.utilities import calculate_average  
  
print(calculate_average([80, 90, 70]))
```

38. Importing a Class from a Package

Suppose:

school/student.py

contains:

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
```

Use:

```
from school.student import Student
```

```
student1 = Student("Dara", 85)
```

39. Complete Package Structure

Example:

```
student_management/
├── main.py
└── school/
    ├── __init__.py
    ├── student.py
    └── utilities.py
```

40. student.py

```
class Student:
    school_name = "Technical Training Center"

    def __init__(self, student_id, name, score):
        self.student_id = student_id
        self.name = name
        self.score = score

    def display_info(self):
        print("ID:", self.student_id)
        print("Name:", self.name)
        print("Score:", self.score)

    def get_result(self):
        if self.score >= 50:
            return "Pass"
        return "Fail"
```

41. utilities.py

```
def calculate_average(scores):  
    if len(scores) == 0:  
        return 0  
  
    return sum(scores) / len(scores)  
  
def display_title():  
    print("STUDENT MANAGEMENT SYSTEM")
```

42. main.py

```
from school.student import Student  
from school.utilities import calculate_average, display_title  
  
display_title()  
  
student1 = Student("ST001", "Dara", 85)  
student2 = Student("ST002", "Sokha", 90)  
  
student1.display_info()  
print("Result:", student1.get_result())  
  
student2.display_info()  
print("Result:", student2.get_result())  
  
scores = [student1.score, student2.score]  
  
print("Average:", calculate_average(scores))
```

43. Relationship Between Class, Module, and Package

The relationship can be represented as:

```
Application  
  ↓  
Package  
  ↓  
Modules  
  ↓  
Classes / Functions  
  ↓  
Objects / Operations
```

Example:

school	→ Package
school/student.py	→ Module
Student	→ Class
student1	→ Object
student1.display_info()	→ Method call

44. Code Reusability

Reusable code can be written once and used in many places.

Example:

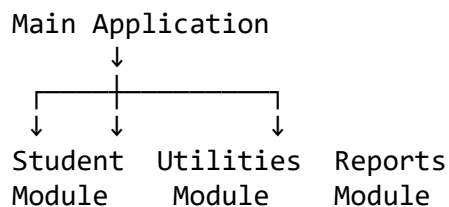
```
def calculate_average(scores):  
    return sum(scores) / len(scores)
```

Instead of rewriting this function in every file, place it in a module and import it when needed.

45. Modular Programming

Modular programming divides an application into logical components.

Example:



Each module has a clear responsibility.

46. Advantages of Modular Programming

Advantages include:

- Better organization.
- Easier maintenance.
- Easier testing.
- Improved code reuse.
- Smaller files.
- Clear responsibilities.
- Easier collaboration.
- Easier debugging.

47. Basic PEP 8 Coding Conventions

PEP 8 provides common Python coding-style recommendations.

At this level, trainees should apply basic conventions such as:

Class Names

Use CapWords:

```
class StudentRecord:  
    pass
```

Function and Variable Names

Use lowercase with underscores:

```
def calculate_average():  
    pass
```

```
student_name = "Dara"
```

Constants

Usually use uppercase:

```
PASS_SCORE = 50
```

Indentation

Use four spaces per indentation level.

Spacing

Recommended:

```
total = a + b
```

rather than:

```
total=a+b
```

48. Documentation and Comments

Comments help explain code.

Example:

```
# Create student object  
student1 = Student("Dara", 85)
```

Functions and classes may also use docstrings.

Example:

```
def calculate_average(scores):  
    """Return the average value of a score list."""  
    return sum(scores) / len(scores)
```

49. Project Folder Organization

An organized project might use:

```
project/  
├── main.py  
├── README.txt  
└── school/  
    ├── __init__.py  
    ├── student.py  
    └── utilities.py
```

Good organization makes files easier to:

- Locate
- Maintain
- Test
- Reuse
- Share

50. Common Class Errors

Missing self

Incorrect:

```
class Student:  
    def display_name():  
        print("Student")
```

For a normal instance method:

```
class Student:  
    def display_name(self):  
        print("Student")
```

51. Incorrect Constructor Name

Incorrect:

```
def init(self, name):
```

Correct:

```
def __init__(self, name):  
    self.name = name
```

The underscores are required.

52. Accessing an Undefined Attribute

Example:

```
class Student:  
    def __init__(self, name):  
        self.name = name
```

```
student1 = Student("Dara")
```

```
print(student1.score)
```

This causes an attribute-related error because score was not defined.

53. Common Module Errors

Incorrect Module Name

If the file is:

```
calculator.py
```

then:

```
import calculate
```

will not import it correctly.

Use:

```
import calculator
```

File Not in Expected Location

Python must be able to find the imported module.

For a beginner project, keep related files in the expected project directory or package structure.

54. Common Package Errors

Possible package problems include:

- Incorrect folder name.
- Incorrect module name.
- Missing required project file.
- Incorrect import path.
- Incorrect indentation.
- Running from an unexpected directory.
- Typographical errors in imports.

55. Debugging Classes, Modules, and Packages

When a modular program does not work:

1. Check class names.
2. Check `__init__()` spelling.
3. Check `self`.
4. Check attribute names.
5. Check method names.
6. Check object creation.
7. Check function parameters.
8. Check module filenames.
9. Check package folder names.
10. Check import statements.
11. Check file locations.
12. Check capitalization.
13. Check indentation.
14. Check syntax errors.
15. Execute individual modules where appropriate.
16. Test classes separately.
17. Test functions separately.
18. Integrate components gradually.
19. Compare actual and expected output.
20. Correct errors and execute again.

SELF-CHECK No. 12.1

Part A — Multiple Choice

1. What is a class?

- A. A blueprint for creating objects
- B. A loop
- C. A string method
- D. A file extension

2. What is an object?

- A. An instance of a class
- B. A Python keyword only
- C. A loop counter
- D. A comment

3. Which keyword creates a class?

- A. def
- B. class
- C. object
- D. new

4. Which method is commonly used to initialize an object?

- A. `init()`
- B. `_init_()`
- C. `__init__()`
- D. `start()`

5. Which parameter normally refers to the current object?

- A. `this`
- B. `self`
- C. `object`
- D. `current`

6. Which syntax creates an object?

- A. `student1 = Student()`
- B. `Student = object`
- C. `create Student`
- D. `new Student`

7. Which syntax accesses an attribute?

- A. student1[name]
- B. student1.name
- C. student1->name
- D. student1:name

8. A function defined inside a class is called a:

- A. Module
- B. Method
- C. Package
- D. Loop

9. A Python module is normally:

- A. A .py file containing reusable code
- B. A database
- C. A hardware component
- D. A loop

10. Which imports the math module?

- A. include math
- B. import math
- C. using math
- D. load math

11. Which imports only sqrt from math?

- A. from math import sqrt
- B. import math.sqrt
- C. load sqrt from math
- D. using sqrt

12. Which uses a module alias?

- A. import math as m
- B. import math = m
- C. alias math m
- D. math as m

13. What is a package primarily used for?

- A. Organizing related modules
- B. Replacing Python
- C. Creating hardware
- D. Removing files

14. Which file is commonly used in a traditional explicit package structure?

- A. package.txt
- B. __init__.py
- C. start.exe
- D. module.ini

15. What is one benefit of modular programming?

- A. Increased duplicate code
- B. Improved code organization and reuse
- C. No testing required
- D. No functions required

ANSWER KEY No. 12

Part A

No.	Answer
1	A
2	A
3	B
4	C
5	B
6	A
7	B
8	B
9	A
10	B
11	A
12	A
13	A
14	B
15	B

OPERATION SHEET No. 12.1

Title: Create and Integrate Python Classes, Modules, and Packages

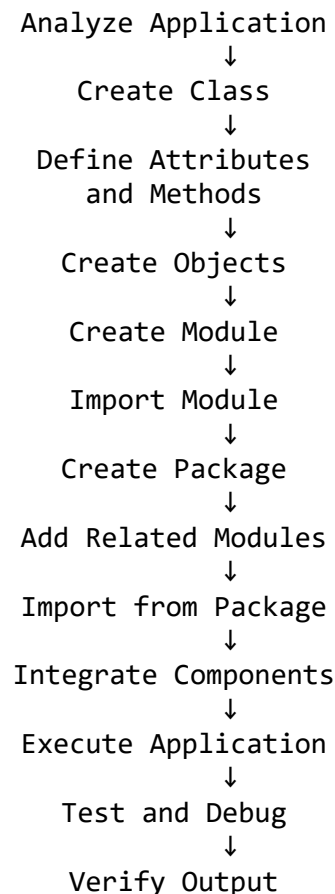
Performance Objective

Given a computer with Python installed and an approved development environment, the trainee must create a class, instantiate objects, define attributes and methods, create reusable modules, import built-in and user-defined modules, create a package, import package modules, integrate the components, execute the application, and verify its output.

Description

This operation demonstrates the complete development sequence for an organized Python application using OOP and modular programming.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1

No.	Description	Unit	Qty
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	File Management Tool	Software	1
7	Python Programming Reference Guide	Copy	1
8	CBLM / Practice Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create project folder	Files are organized correctly
2. Open Python IDE	Environment opens successfully
3. Create class	Correct class syntax is applied
4. Define <code>__init__()</code>	Object initialization is correct
5. Define attributes	Required data fields are created
6. Define methods	Required operations are implemented
7. Instantiate objects	Objects are successfully created
8. Access attributes	Correct values are retrieved
9. Modify attributes	Object data changes correctly
10. Invoke methods	Required behavior executes
11. Create module	Reusable .py file is created
12. Import built-in module	Import statement is correct
13. Import user-defined module	Reusable code executes
14. Create package directory	Package structure is correct
15. Add <code>__init__.py</code>	Explicit package structure is established
16. Add modules to package	Related code is organized
17. Import from package	Correct import path is used
18. Integrate application	Components work together
19. Apply basic naming conventions	Code is readable and consistent
20. Execute application	Program runs successfully
21. Debug identified errors	Corrections are applied

Procedure/Step	Key Performance
22. Verify output	Results match requirements

Assessment Method

- Direct observation
- Practical demonstration
- Project-file inspection
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 12.1

Title: Create Classes and Objects

Performance Task

Create a Python class, define attributes, instantiate objects, and access object data.

Performance Objective

Given Python IDLE/IDE, the trainee must create a class using correct syntax, apply `__init__()`, create at least two objects, and display their attributes.

Description

The trainee practices fundamental OOP concepts.

Picture/Overview

```
Define Class
  ↓
Create __init__()
  ↓
Define Attributes
  ↓
Create Object 1
Create Object 2
  ↓
Access Attributes
  ↓
Verify Results
```

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Class Exercise File	Set	1
5	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
----------------	-----------------

Procedure/Step	Key Performance
1. Define class	Correct class syntax is used
2. Define <code>__init__()</code>	Constructor is correctly spelled
3. Use <code>self</code>	Instance attributes are defined
4. Create first object	Object is successfully instantiated
5. Create second object	Independent object is created
6. Access attributes	Correct values are displayed
7. Verify object data	Values match supplied arguments

Practical Exercise

```
class Student:
    def __init__(self, student_id, name, score):
        self.student_id = student_id
        self.name = name
        self.score = score
```

```
student1 = Student("ST001", "Dara", 85)
student2 = Student("ST002", "Sokha", 90)
```

```
print(student1.student_id)
print(student1.name)
print(student1.score)
```

```
print(student2.student_id)
print(student2.name)
print(student2.score)
```

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 12.2

Title: Define and Use Class Attributes and Methods

Performance Task

Create object attributes and class methods, modify object data, and execute object behavior.

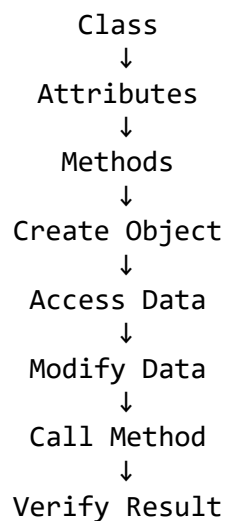
Performance Objective

Given a student class, the trainee must define methods, access and modify attributes, and correctly invoke class methods.

Description

The trainee expands a basic class into a functional object.

Picture/Overview



Practical Exercise

```
class Student:
    school_name = "Technical Training Center"

    def __init__(self, name, score):
        self.name = name
        self.score = score

    def display_info(self):
        print("Name:", self.name)
        print("Score:", self.score)

    def get_result(self):
```

```
if self.score >= 50:  
    return "Pass"  
return "Fail"
```

```
student1 = Student("Dara", 85)
```

```
student1.display_info()
```

```
print("Result:", student1.get_result())
```

```
student1.score = 90
```

```
print("New Score:", student1.score)
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Define class variable	Shared value is correctly created
2. Define instance attributes	Object data is initialized
3. Define display method	Required information is shown
4. Define result method	Correct value is returned
5. Instantiate object	Object is created
6. Call methods	Methods execute correctly
7. Modify attribute	New value is stored
8. Verify results	Updated object behaves correctly

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 12.2

Title: Create and Import Python Modules

Performance Task

Create a reusable module containing functions and import the module into a main application.

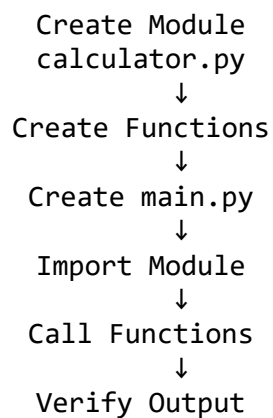
Performance Objective

Given a project folder, the trainee must create a user-defined module, import it using different import statements, and execute imported functions.

Description

The trainee practices modular programming using multiple .py files.

Picture/Overview



File 1 — calculator.py

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
def multiply(a, b):  
    return a * b
```

File 2 — main.py

Method A

```
import calculator

print(calculator.add(5, 10))
```

Method B

```
from calculator import subtract

print(subtract(20, 5))
```

Method C

```
import calculator as calc

print(calc.multiply(5, 10))
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create calculator.py	Module file is correctly named
2. Create functions	Reusable functions operate correctly
3. Create main.py	Application file is created
4. Apply import	Module namespace is used correctly
5. Apply from ... import	Specific function is imported
6. Apply import ... as	Alias works correctly
7. Execute imported functions	Correct results are returned
8. Verify output	Results meet requirements

Assessment Method

- Direct observation
- File inspection
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 12.3

Title: Create a Module Containing a Python Class

Performance Task

Create a reusable module containing a class and instantiate objects from the imported class.

Performance Objective

Given a project folder, the trainee must create a module containing a class, import the class into a separate Python file, create an object, and execute its methods.

File Structure

```
class_module_project/  
├── main.py  
└── student.py
```

student.py

```
class Student:  
    def __init__(self, name, score):  
        self.name = name  
        self.score = score  
  
    def display_info(self):  
        print("Name:", self.name)  
        print("Score:", self.score)  
  
    def get_result(self):  
        return "Pass" if self.score >= 50 else "Fail"
```

main.py

```
from student import Student  
  
student1 = Student("Dara", 85)  
  
student1.display_info()  
  
print("Result:", student1.get_result())
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create class module	File structure is correct

Procedure/Step	Key Performance
2. Define class	Class is reusable
3. Define constructor	Object initialization works
4. Define methods	Required behavior is provided
5. Import class	Correct import syntax is used
6. Create object	Imported class instantiates correctly
7. Execute methods	Imported behavior works
8. Verify output	Results are correct

Assessment Method

- Direct observation
- Practical exercise
- Project-file inspection
- Output verification

TASK SHEET No. 12.4

Title: Import and Use Built-In Python Modules

Performance Task

Import and execute components from Python built-in modules.

Performance Objective

Given Python and a development environment, the trainee must apply `import`, `from ... import`, and aliases to built-in modules.

Practical Exercise A — `math`

```
import math

print(math.sqrt(25))
print(math.pi)
```

Practical Exercise B — Specific Import

```
from math import sqrt

print(sqrt(81))
```

Practical Exercise C — Alias

```
import math as m

print(m.pow(2, 3))
```

Practical Exercise D — `random`

```
import random

number = random.randint(1, 10)

print(number)
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Import built-in module	Module loads correctly
2. Access module member	Dot notation is correct
3. Import specific function	Function executes directly
4. Apply alias	Alias references module correctly
5. Execute program	No import errors occur

Procedure/Step	Key Performance
6. Verify output	Results are reasonable/correct

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 12.5

Title: Create and Import a Python Package

Performance Task

Create a package containing related Python modules and import package components into a main application.

Performance Objective

Given a project requirement, the trainee must create an organized package directory, add related modules, apply appropriate imports, and execute package components successfully.

Package Structure

```
student_management/  
├── main.py  
└── school/  
    ├── __init__.py  
    ├── student.py  
    └── utilities.py
```

school/student.py

```
class Student:  
    def __init__(self, student_id, name, score):  
        self.student_id = student_id  
        self.name = name  
        self.score = score  
  
    def display_info(self):  
        print("ID:", self.student_id)  
        print("Name:", self.name)  
        print("Score:", self.score)
```

school/utilities.py

```
def calculate_average(scores):  
    if len(scores) == 0:  
        return 0  
  
    return sum(scores) / len(scores)
```

main.py

```
from school.student import Student  
from school.utilities import calculate_average
```

```

student1 = Student("ST001", "Dara", 85)
student2 = Student("ST002", "Sokha", 90)

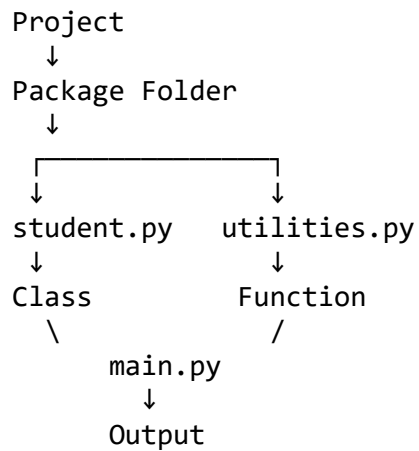
student1.display_info()
student2.display_info()

scores = [student1.score, student2.score]

print("Average:", calculate_average(scores))

```

Picture/Overview



Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create project directory	Correct folder is created
2. Create package directory	Related modules are grouped
3. Create <code>__init__.py</code>	Explicit package structure is present
4. Create class module	Class is correctly stored
5. Create utility module	Function is reusable
6. Create <code>main.py</code>	Application entry file exists
7. Import class from package	Correct import path is applied
8. Import function from package	Correct import path is applied
9. Create objects	Imported class works
10. Execute imported function	Correct result is returned
11. Verify folder organization	Project structure is clear
12. Verify output	Program operates correctly

Assessment Method

- Direct observation
- Folder/file inspection
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 12.6

Title: Integrate Classes, Modules, and Packages

Performance Task

Integrate multiple Python components into one organized application.

Performance Objective

Given class, module, and package requirements, the trainee must create separate reusable components and integrate them through a main Python program.

Description

This task prepares the trainee for the final Job Sheet.

Required Structure

```
training_app/  
├── main.py  
└── training/  
    ├── __init__.py  
    ├── trainee.py  
    └── calculations.py
```

trainee.py

Create a Trainee class containing:

```
trainee_id  
name  
scores
```

Methods:

```
display_info()  
get_average()  
get_result()
```

calculations.py

Create reusable functions:

```
calculate_total()  
calculate_average()
```

main.py

Import and use both components.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Analyze components	Classes/functions are identified
2. Create package	Folder structure is correct
3. Create class module	OOP component works
4. Create calculation module	Reusable functions work
5. Create main application	Integration logic is correct
6. Import class	Import succeeds
7. Import functions	Imports succeed
8. Create objects	Data is initialized
9. Execute methods/functions	Components cooperate
10. Test integration	Results are verified

Assessment Method

- Direct observation
- Project inspection
- Source-code review
- Program execution
- Output verification

JOB SHEET No. 12.1

Title: Develop a Modular Student Management Application

Performance Task

Develop a complete Python application that integrates classes, objects, modules, and packages.

Performance Objective

Given a computer, Python development environment, project requirements, and reference materials, the trainee must analyze the application, create an organized project folder, define reusable classes and functions, create modules and a package, import components correctly, instantiate objects, execute methods and functions, test the application, debug errors, and verify expected results.

Description

Develop a project named:

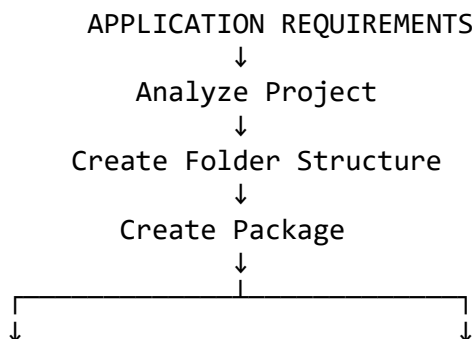
student_management_app

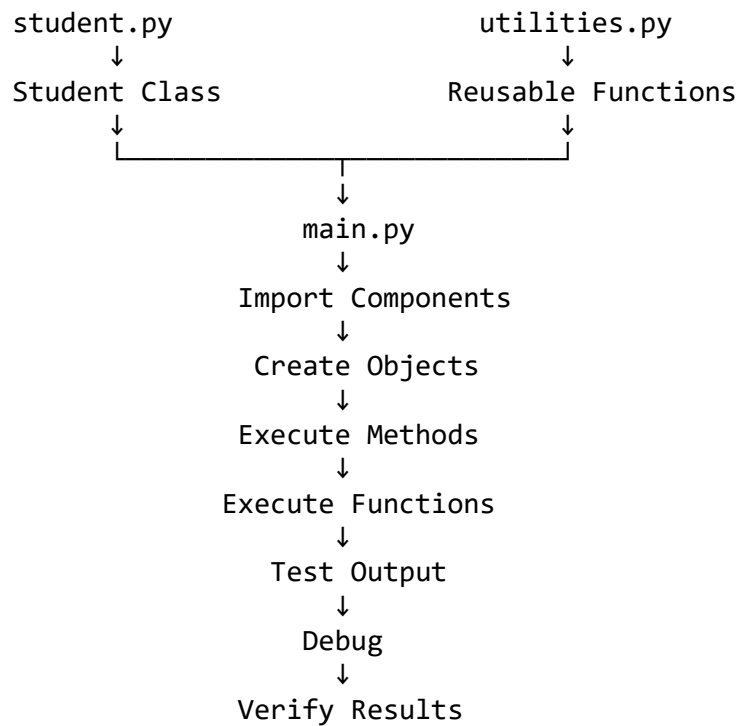
The project must demonstrate all major competencies of LO12.

Required Project Structure

```
student_management_app/  
├── main.py  
└── school/  
    ├── __init__.py  
    ├── student.py  
    └── utilities.py
```

Picture/Overview





Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	File Management Tool	Software	1
7	Python Programming Reference Guide	Copy	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Analyze application requirements	Required components are identified
2. Create project folder	Project is organized correctly

Procedure/Step	Key Performance
3. Create package folder	school directory exists
4. Create <code>__init__.py</code>	Package is explicitly structured
5. Create <code>student.py</code>	Class module is created
6. Define <code>Student</code> class	Correct class syntax is used
7. Define class variable	Shared value is created
8. Define <code>__init__()</code>	Objects initialize correctly
9. Define instance attributes	Required student data is stored
10. Define class methods	Required behaviors are implemented
11. Create <code>utilities.py</code>	Reusable function module exists
12. Define utility functions	Functions return correct results
13. Create <code>main.py</code>	Main application is created
14. Import class from package	Import succeeds
15. Import functions from package	Imports succeed
16. Instantiate multiple objects	Student objects are created
17. Access attributes	Object information is available
18. Modify attribute	Object data can be updated
19. Call methods	Object behavior executes
20. Execute utility functions	Module functions return correct results
21. Apply coding conventions	Names/indentation are appropriate
22. Add comments/docstrings	Code is understandable
23. Execute complete application	Program runs successfully
24. Test multiple student records	Different data is verified
25. Identify import/class errors	Problems are recognized
26. Debug application	Errors are corrected
27. Verify final output	Results meet requirements

Part 1 — school/student.py

Create:

```
class Student:
    """Represent a student record."""

    school_name = "Technical Training Center"
```

```

def __init__(self, student_id, name, course, scores):
    self.student_id = student_id
    self.name = name
    self.course = course
    self.scores = scores

def display_info(self):
    print("ID:", self.student_id)
    print("Name:", self.name)
    print("Course:", self.course)
    print("Scores:", self.scores)

def get_average(self):
    if len(self.scores) == 0:
        return 0

    return sum(self.scores) / len(self.scores)

def get_result(self):
    if self.get_average() >= 50:
        return "Pass"

    return "Fail"

def update_course(self, new_course):
    self.course = new_course

```

Part 2 — school/utilities.py

Create:

```

def calculate_total(scores):
    """Return the sum of all scores."""
    return sum(scores)

def calculate_average(scores):
    """Return the average score."""
    if len(scores) == 0:
        return 0

    return sum(scores) / len(scores)

def display_header():
    """Display application title."""
    print("=====")

```

```
print("          STUDENT MANAGEMENT SYSTEM")
print("=====")
```

Part 3 — school/init.py

For this basic exercise, the file may remain empty:

School package

Part 4 — main.py

Create:

```
from school.student import Student
from school.utilities import (
    calculate_total,
    calculate_average,
    display_header,
)
```

```
display_header()
```

```
student1 = Student(
    "ST001",
    "Dara",
    "AI Basic Python Developer",
    [80, 85, 90],
)
```

```
student2 = Student(
    "ST002",
    "Sokha",
    "AI Basic Python Developer",
    [70, 75, 80],
)
```

```
student3 = Student(
    "ST003",
    "Vanna",
    "Python",
    [40, 45, 50],
)
```

```
print("\nSTUDENT 1")
student1.display_info()
```

```

print("Average:", student1.get_average())
print("Result:", student1.get_result())

print("\nSTUDENT 2")
student2.display_info()
print("Average:", student2.get_average())
print("Result:", student2.get_result())

print("\nSTUDENT 3")
student3.display_info()
print("Average:", student3.get_average())
print("Result:", student3.get_result())

print("\nMODIFY OBJECT ATTRIBUTE")
student3.update_course("AI Basic Python Developer")
print("New Course:", student3.course)

print("\nUTILITY MODULE FUNCTIONS")

scores = student1.scores

print("Total:", calculate_total(scores))
print("Average:", calculate_average(scores))

print("\nCLASS VARIABLE")
print("School:", Student.school_name

```

Expected Main Output

```

=====
          STUDENT MANAGEMENT SYSTEM
=====

STUDENT 1
ID: ST001
Name: Dara
Course: AI Basic Python Developer
Scores: [80, 85, 90]
Average: 85.0
Result: Pass

STUDENT 2

```

ID: ST002
Name: Sokha
Course: AI Basic Python Developer
Scores: [70, 75, 80]
Average: 75.0
Result: Pass

STUDENT 3
ID: ST003
Name: Vanna
Course: Python
Scores: [40, 45, 50]
Average: 45.0
Result: Fail

MODIFY OBJECT ATTRIBUTE
New Course: AI Basic Python Developer

UTILITY MODULE FUNCTIONS
Total: 255
Average: 85.0

CLASS VARIABLE
School: Technical Training Center

Testing Requirements

Test Item	Expected Result	Actual Result	Pass/Fail
Student 1 ID	ST001	_____	_____
Student 1 name	Dara	_____	_____
Student 1 average	85.0	_____	_____
Student 1 result	Pass	_____	_____
Student 2 average	75.0	_____	_____
Student 2 result	Pass	_____	_____
Student 3 average	45.0	_____	_____
Student 3 result	Fail	_____	_____
Updated course	AI Basic Python Developer	_____	_____
Utility total	255	_____	_____
Utility average	85.0	_____	_____
Class variable	Technical Training Center	_____	_____

Debugging Requirements

Error 1 — Incorrect Constructor

Incorrect:

```
class Student:
    def init(self, name):
        self.name = name
```

Correct:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Error 2 — Missing self

Incorrect:

```
class Student:
    def display_info():
        print("Student")
```

Correct:

```
class Student:
    def display_info(self):
        print("Student")
```

Error 3 — Wrong Module Name

If the module is:

student.py

Incorrect:

```
from students import Student
```

Correct:

```
from student import Student
```

when both files are in the same directory.

For the package used in the Job Sheet:

```
from school.student import Student
```

Error 4 — Wrong Package Import

Incorrect:

```
from student import Student
```

when `student.py` is located inside:

`school/`

Correct:

```
from school.student import Student
```

Error 5 — Undefined Attribute

Incorrect:

```
print(student1.address)
```

when no `address` attribute exists.

The trainee must either define the attribute or use an existing one.

Assessment Method

- Oral questioning
- Written/theoretical assessment
- Practical workshop assessment
- Direct observation
- Project structure inspection
- Source-code inspection
- Program execution
- Output verification
- Debugging demonstration

WORKSHEET No. 12.1

Title: Python Class, Module, and Package Practice Worksheet

Trainee Name: _____

Date: _____

Part A — Identify OOP Components

Complete:

Term	Description
Class	_____
Object	_____
Attribute	_____
Method	_____
Constructor	_____

Part B — Complete a Class

```
_____ Student:  
    def _____(self, name):  
        self.name = name
```

Answers:

Part C — Create an Object

Given:

```
class Student:  
    def __init__(self, name):  
        self.name = name
```

Create an object containing "Dara":

Part D — Access an Attribute

Write code to display:

student1.name

Answer:

Part E — Modify an Attribute

Change:

`student1.name`

to:

Sokha

Answer:

Part F — Define a Method

Complete:

```
class Student:
    def __init__(self, name):
        self.name = name

    def _____(self):
        print(self.name)
```

Use method name:

`display_name`

Part G — Module Imports

Complete:

Import math:

Import sqrt from math:

Import math as m:

Part H — User-Defined Module

Given a module:

calculator.py

containing:

```
def add(a, b):  
    return a + b
```

Write code in main.py to import and execute add(5, 10).

Part I — Package Structure

Fill the missing file:

```
school/  
├── _____  
├── student.py  
└── utilities.py
```

Answer:

Part J — Import from Package

Given:

school/student.py

containing a class named:

Student

Write the import:

Part K — Class Variable vs Instance Variable

Given:

```
class Student:  
    school = "Training Center"  
  
    def __init__(self, name):  
        self.name = name
```

Class variable: _____

Instance variable: _____

Part L — PEP 8 Naming

Which is more appropriate for a class?

- A. student_record
- B. StudentRecord

Answer: _____

Which is more appropriate for a function?

- A. CalculateAverage
- B. calculate_average

Answer: _____

Part M — Debugging

What is wrong?

```
class Student:
    def init(self, name):
        self.name = name
```

Answer:

Correct it:

PERFORMANCE CHECKLIST No. 12

Title: Develop Program Using Class, Module and Package

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Identified the concept of OOP	☐	☐
2	Identified a Python class	☐	☐
3	Identified an object	☐	☐
4	Identified attributes	☐	☐
5	Identified methods	☐	☐
6	Identified a Python module	☐	☐
7	Identified a Python package	☐	☐
8	Created a class using correct syntax	☐	☐
9	Applied appropriate class naming	☐	☐
10	Defined <code>__init__()</code> correctly	☐	☐
11	Applied <code>self</code> correctly	☐	☐
12	Defined instance attributes	☐	☐
13	Defined a class variable	☐	☐
14	Defined class methods	☐	☐
15	Created an object from a class	☐	☐
16	Created multiple objects where required	☐	☐
17	Accessed object attributes	☐	☐
18	Modified an object attribute	☐	☐
19	Called object methods	☐	☐
20	Returned values from methods where required	☐	☐
21	Created a reusable user-defined module	☐	☐
22	Added functions to a module	☐	☐
23	Added a class to a module	☐	☐
24	Imported a built-in module	☐	☐
25	Applied <code>import module</code> correctly	☐	☐
26	Applied <code>from ... import</code> correctly	☐	☐
27	Applied <code>import ... as</code> correctly	☐	☐
28	Executed an imported function	☐	☐
29	Instantiated an imported class	☐	☐
30	Created a Python package folder	☐	☐
31	Created <code>__init__.py</code> for the training package	☐	☐
32	Organized related modules inside the package	☐	☐
33	Imported a module from a package	☐	☐
34	Imported a function from a package module	☐	☐
35	Imported a class from a package module	☐	☐

No.	Criteria	Yes	No
36	Integrated class and module components	☐	☐
37	Integrated class, module, and package components	☐	☐
38	Organized project files correctly	☐	☐
39	Applied lowercase-with-underscores for variables/functions	☐	☐
40	Applied CapWords naming for classes	☐	☐
41	Applied four-space indentation	☐	☐
42	Added useful comments/docstrings	☐	☐
43	Saved Python files with .py extension	☐	☐
44	Executed the application successfully	☐	☐
45	Tested object creation	☐	☐
46	Tested attribute access	☐	☐
47	Tested attribute modification	☐	☐
48	Tested method execution	☐	☐
49	Tested module imports	☐	☐
50	Tested package imports	☐	☐
51	Identified class syntax errors	☐	☐
52	Identified <code>__init__()</code> errors	☐	☐
53	Identified <code>self</code> errors	☐	☐
54	Identified module import errors	☐	☐
55	Identified package path errors	☐	☐
56	Applied appropriate debugging techniques	☐	☐
57	Corrected identified errors	☐	☐
58	Verified final output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____